

Agenda:

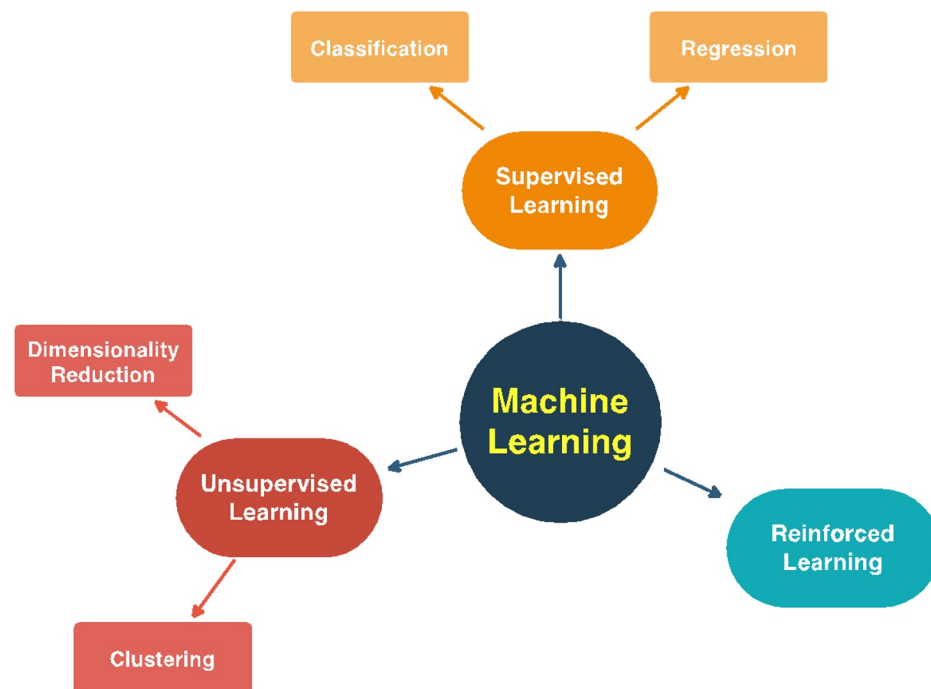
1. Recurrent Neural Networks (RNN)
2. Hands on practice

Machine Learning subcategories:

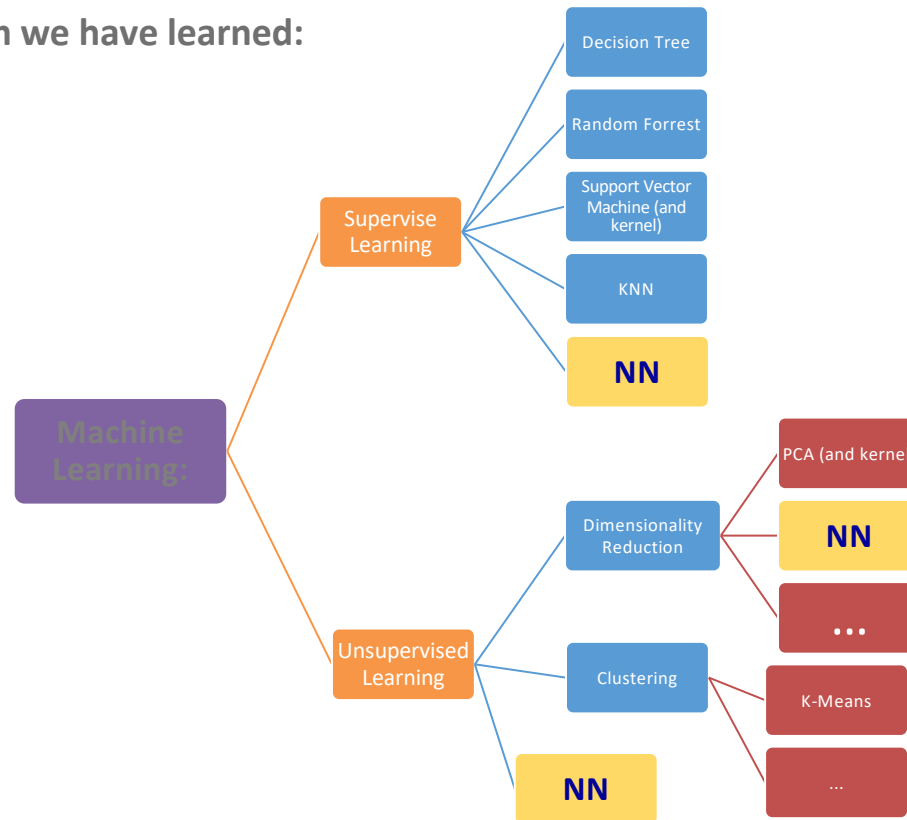
- I. Supervised Learning
- II. Un-supervised Learning
- III. Reinforced learning
- IV. Others...

Where is NN:

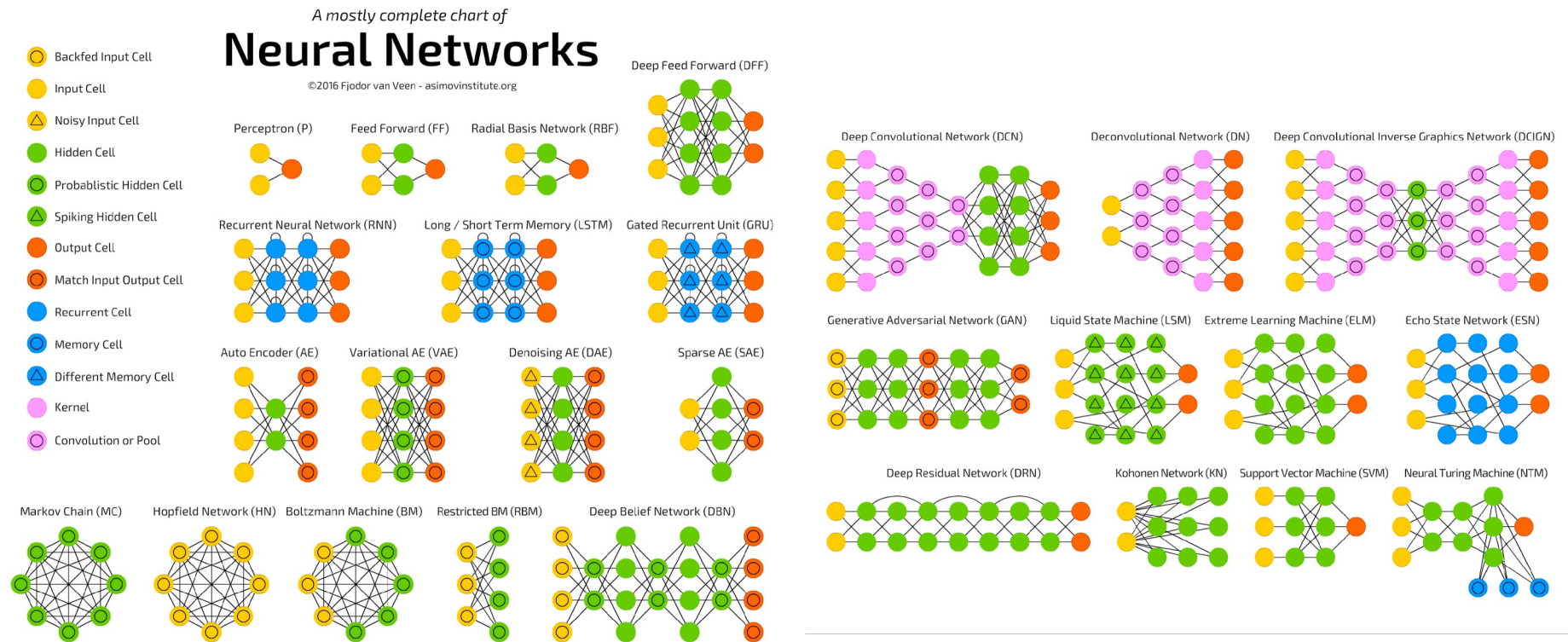
Almost all of them



What algorithm we have learned:



Different structures of Neural Networks:



Recurrent Neural Networks

What are RNN:

Recurrent Neural networks are networks similar to ANN but with internal memory and can study temporal dynamic behaviour.

i.e RNN can remember things from the past!! And so perfect for time series problem!

RNN can work with time sequences varying in size (unlike other NN that get a fixed size input and generate fixed size output), more on this later.

What is the next number of this sequence:

1 2 3 4 5 1 2 3 4 5 ?

In order to properly predict the RNN should remember the sequence from previous data,

Another example:

It is raining outside so I need to pick up an umbrella ...

The NN should remember the key word “raining” from past,

Applications of RNN:

Chatbots

Machine translation

Signal processing, and noise cancelation

Time series prediction (prices, stocks)

Anomaly detection

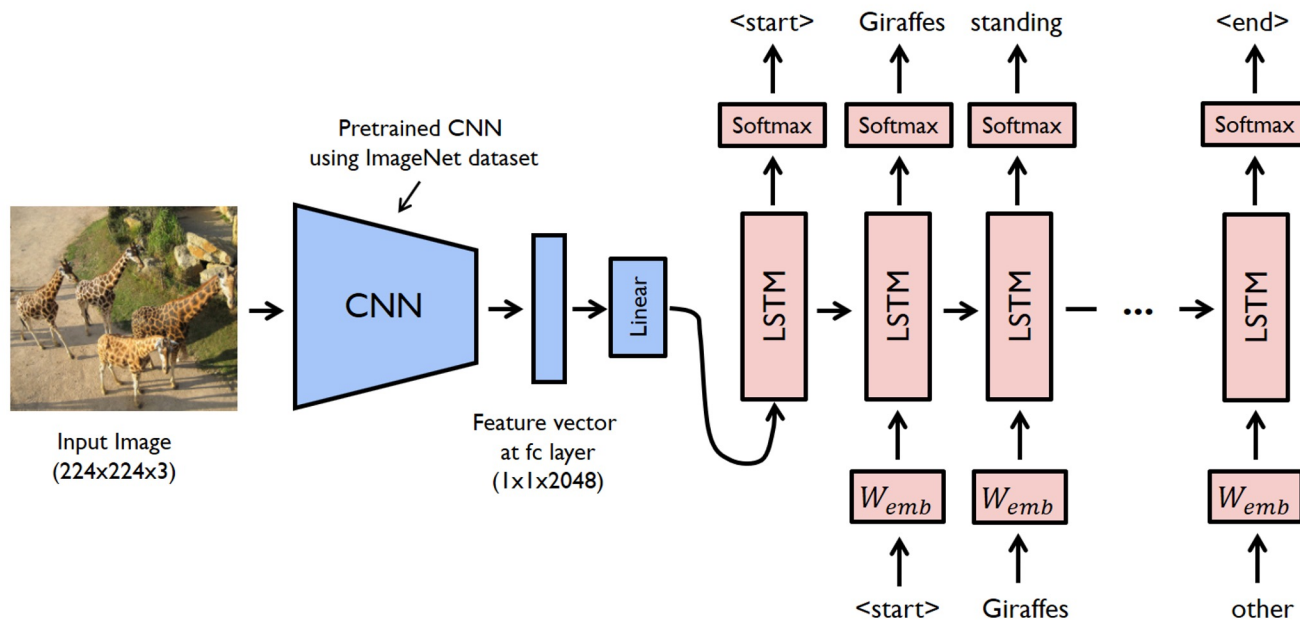
Audio and video recognition (image captioning)

Human-robot interactions (chatbots)

...

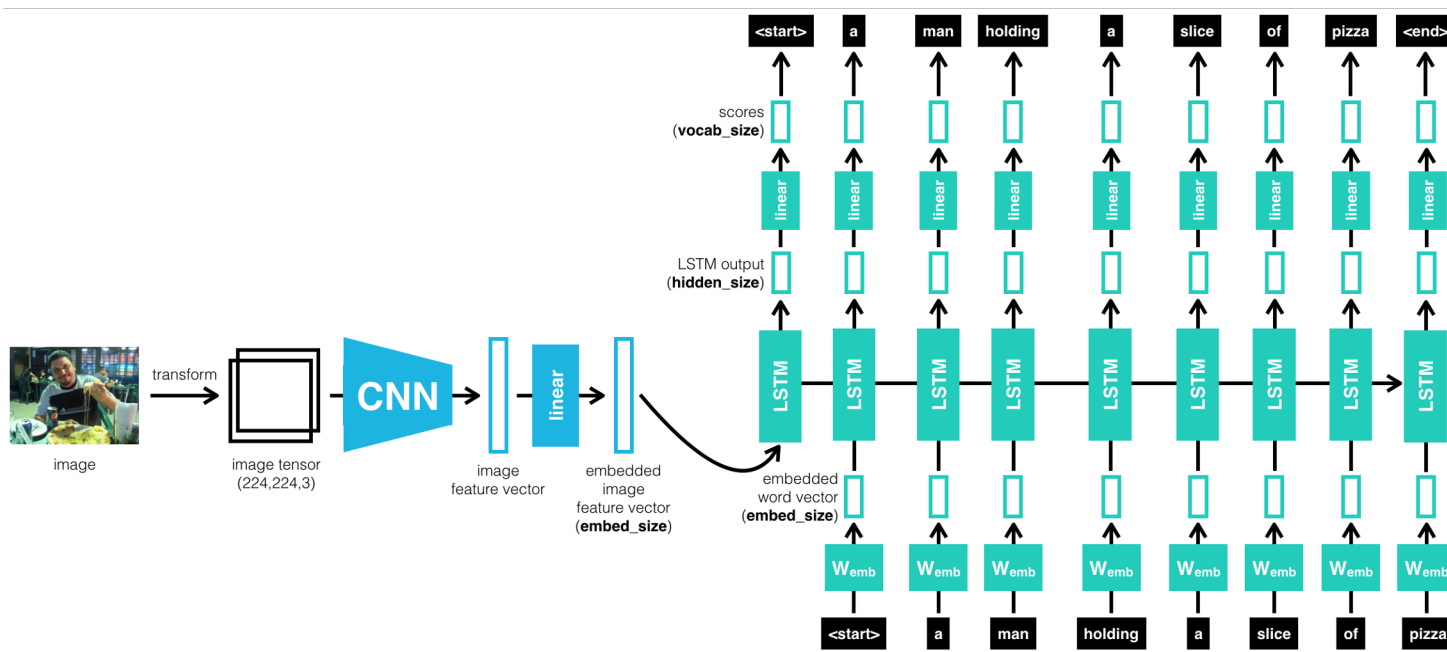
RNN Applications:

One important application is Human-machine interaction, an example is image captioning



RNN Applications:

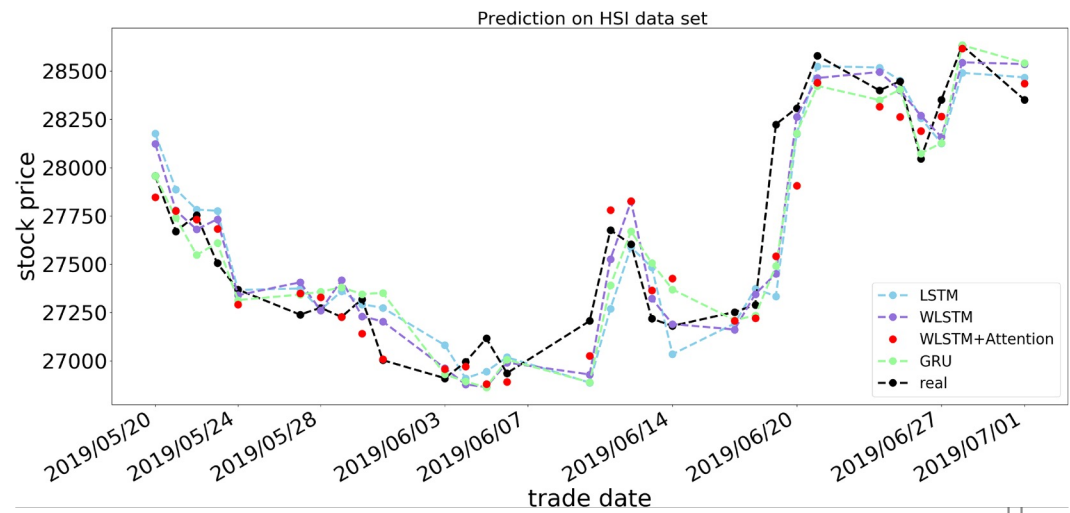
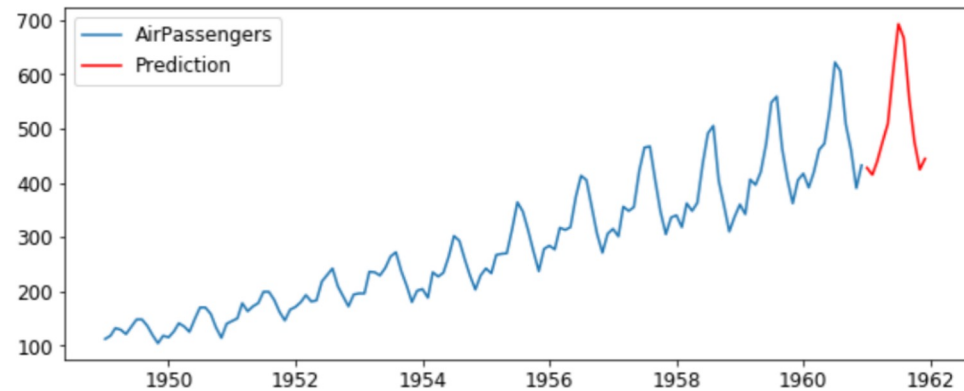
One important application is Human-machine interaction, an example is image captioning



RNN Applications:

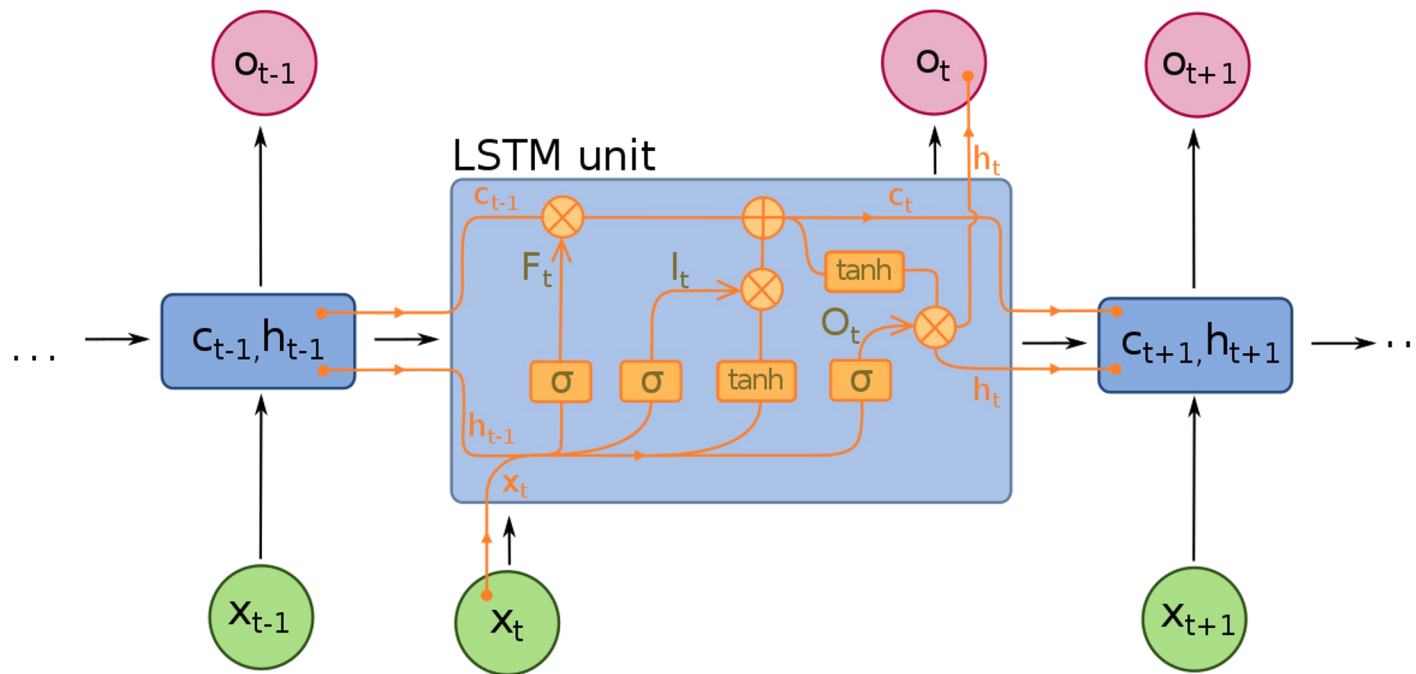
Time series predictions

RNN can be use to predict or classify temporal data like, price, sale...



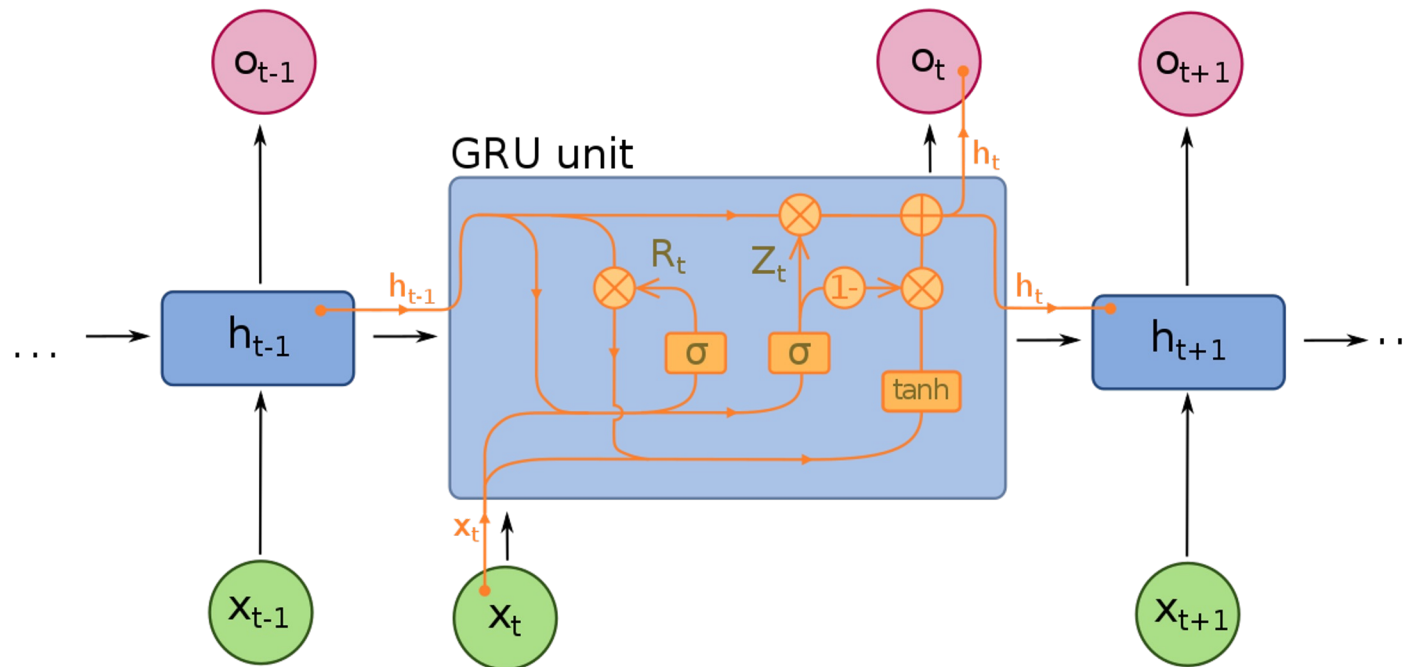
Different units of RNN:

1. LSTM (Long Short Term Memory)



Different units of RNN:

2. GRU (Gated recurrent unit)

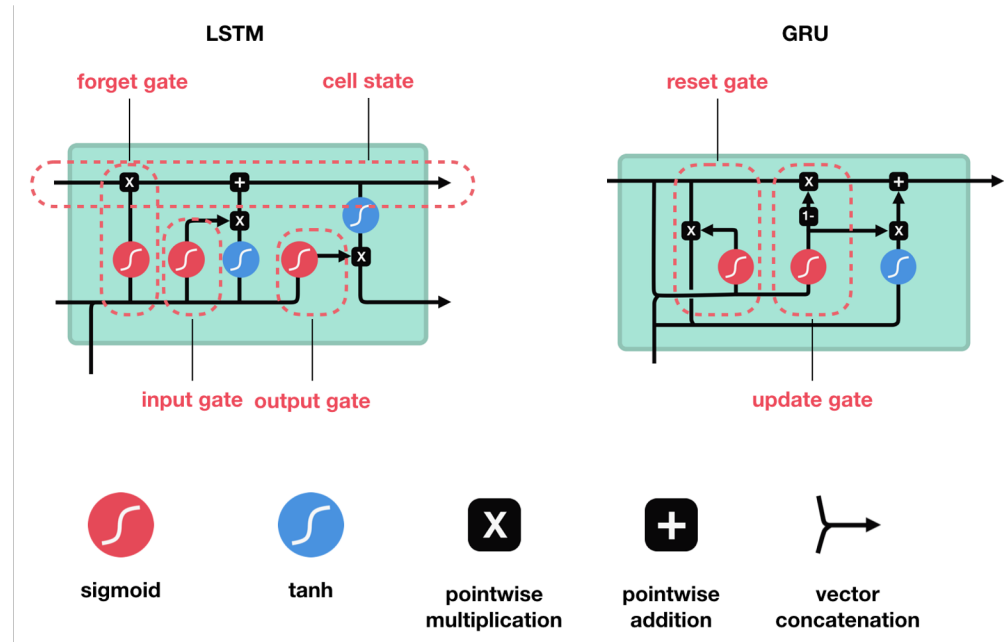


Different units of RNN:

LSTM vs GRU (Gated recurrent unit)

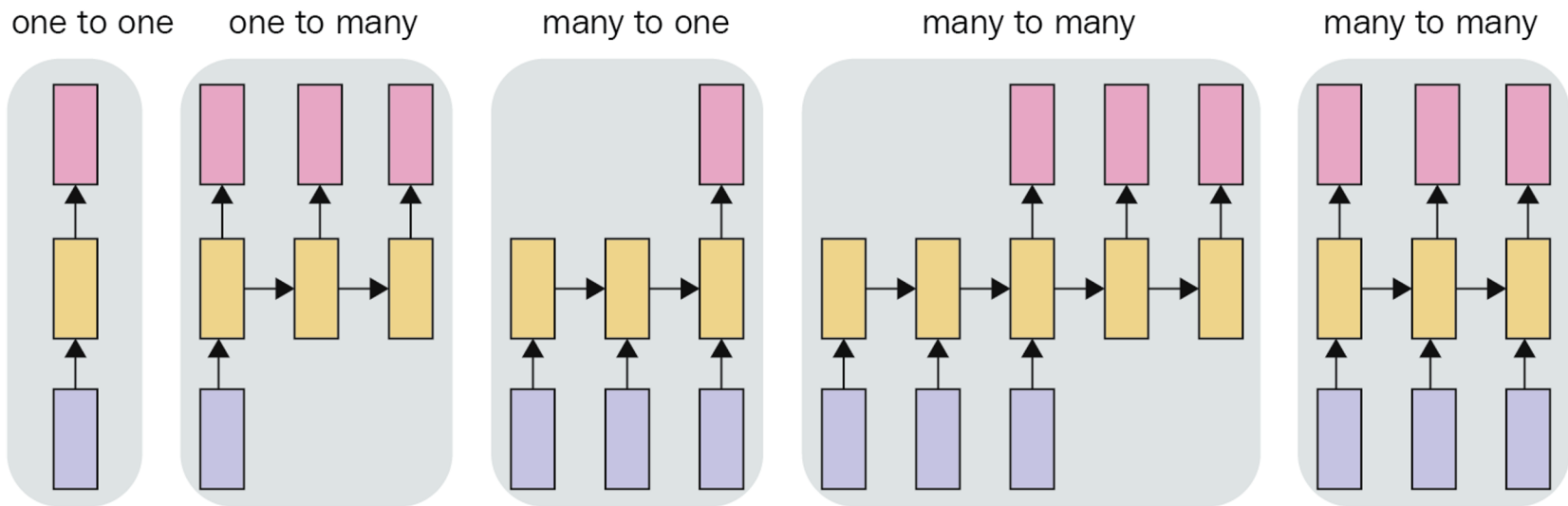
GRU has two gates: reset and update gates

LSTM has three gates: input, output and forget gates



Some of RNN properties:

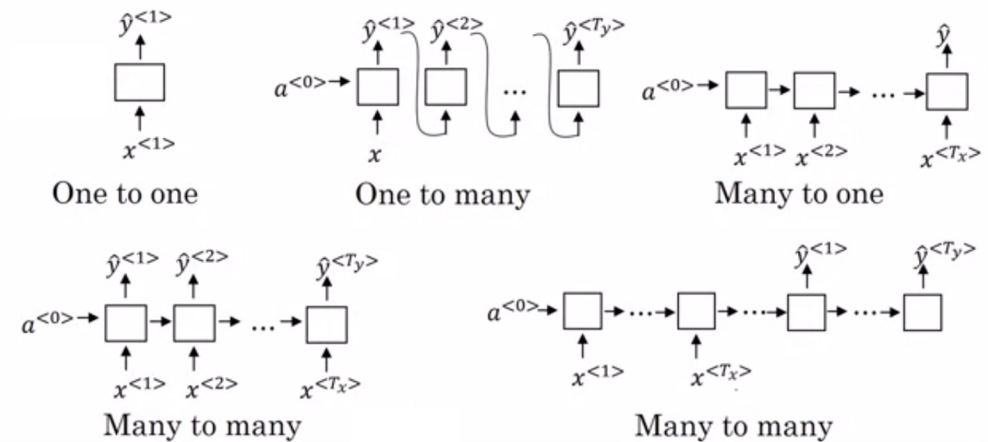
1. Can work with varying size sequences
2. Can work with sequences of vectors



Some of RNN properties:

RNN can work with sequence of vectors

- One to Many, (sequence output), example: Image captioning, where a RNN can take an image and describe it
- Many to one, (sequence input), example: sentiment analysis, like when we classify reviews (a long review will be classified into good or bad)
- Many to Many, (sequence input and output), like translation, read a paragraph and translate into another language



Math behind RNN,

As we have discussed briefly, RNN comes into play when dealing with sequences or time series.

For example:

Inputs: ["It", "is", "raining", "I", "need", "an", _].

Expected Output: "umbrella"

Other examples?

**Math behind RNN,
Forward Propagation**

(no activation function)

X — Input vector

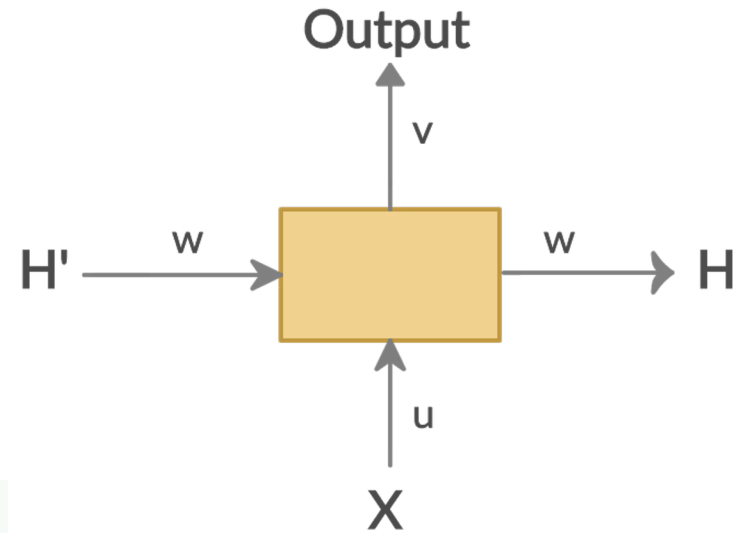
Output — Predicted Output vector

H' — Previous Input's output to be fed in the current input

H — Current input's output to be fed in the next input

$u/v/w$ — Weights in the respective location

$$H = uX + wH'$$
$$\text{Output} = vH$$



Math behind RNN, Backward Prop.

Error term:

$$E = \frac{(\text{Output} - Y)^2}{2}$$

$$\frac{\partial E}{\partial u} = ?, \quad \frac{\partial E}{\partial w} = ?, \quad \frac{\partial E}{\partial v} = ?$$

**Math behind RNN,
Backward Prop.**

Chain Rule

$$\begin{aligned} H &= uX + wH' \\ \text{Output} &= vH \end{aligned} \quad (1)$$

$$\frac{\partial E}{\partial w} = \frac{\partial E}{\partial \text{Output}} \frac{\partial \text{Output}}{\partial H} \frac{\partial H}{\partial w} = (\text{Output} - Y) (v) (H') \quad (2)$$

$$\frac{\partial E}{\partial v} = \frac{\partial E}{\partial \text{Output}} \frac{\partial \text{Output}}{\partial v} = (\text{Output} - Y) \cdot (H) = (\text{Output} - Y) \cdot (uX + wH') \quad (3)$$

$$\frac{\partial E}{\partial u} = \frac{\partial E}{\partial \text{Output}} \frac{\partial \text{Output}}{\partial H} \frac{\partial H}{\partial u} = (\text{Output} - Y) \cdot u \cdot X \quad (4)$$

And update the weights:

$$\begin{aligned} &\therefore \\ w_{new} &= w_{old} - \frac{\partial E}{\partial w}, \quad v_{new} = v_{old} - \frac{\partial E}{\partial v}, \quad u_{new} = u_{old} - \frac{\partial E}{\partial u} \end{aligned}$$

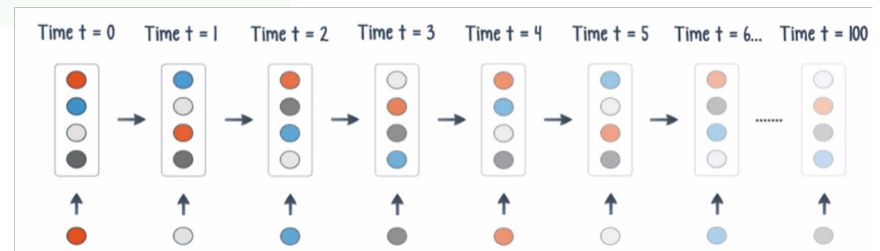
The problem of vanishing Gradients:

$H' \lll 1$ — multiplying by numbers less than 1 values keep decreasing until it eventually tend toward zero. This problem is referred to as **vanishing gradient**.

$H' \ggg 1$ — multiplying by numbers larger than 1 values keep increasing until it eventually tend toward infinity. This problem is referred to as **exploding gradient**.

Vanishing gradient causes that some NN to easily forgot the information from the past!

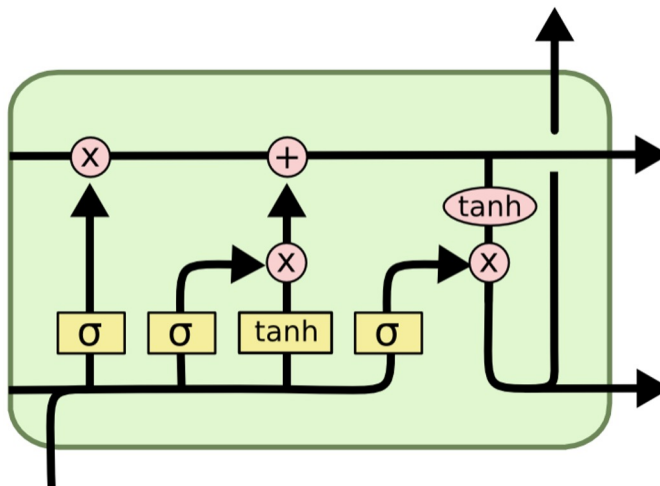
$$\frac{\partial E}{\partial w} = \frac{\partial E}{\partial Output} \frac{\partial Output}{\partial H} \frac{\partial H}{\partial w} = (Output - Y) (v) (H')$$



Math behind LSTM,

Unlike traditional RNN units, LSTM units don't suffer from the Vanishing Gradient problem.

LSTM stand for Long Short Term Memory, these units have global state which is maintained among all the inputs.



Math behind LSTM,

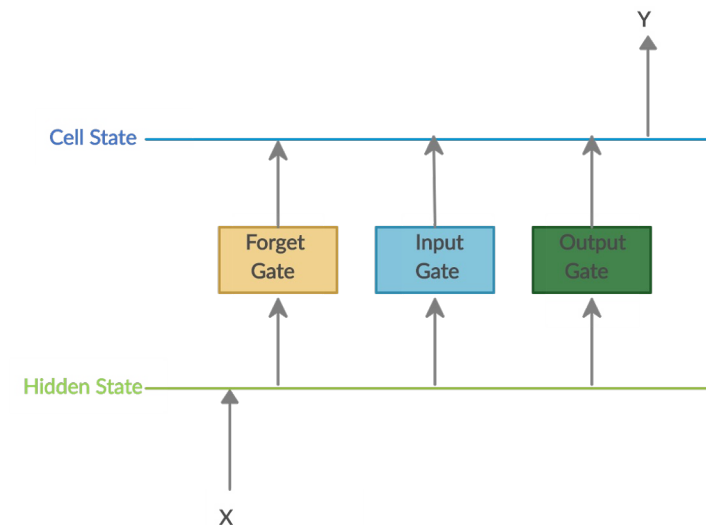
Cell State: the memory layer which contains the context across the inputs.

Hidden State: Simply the hidden layer.

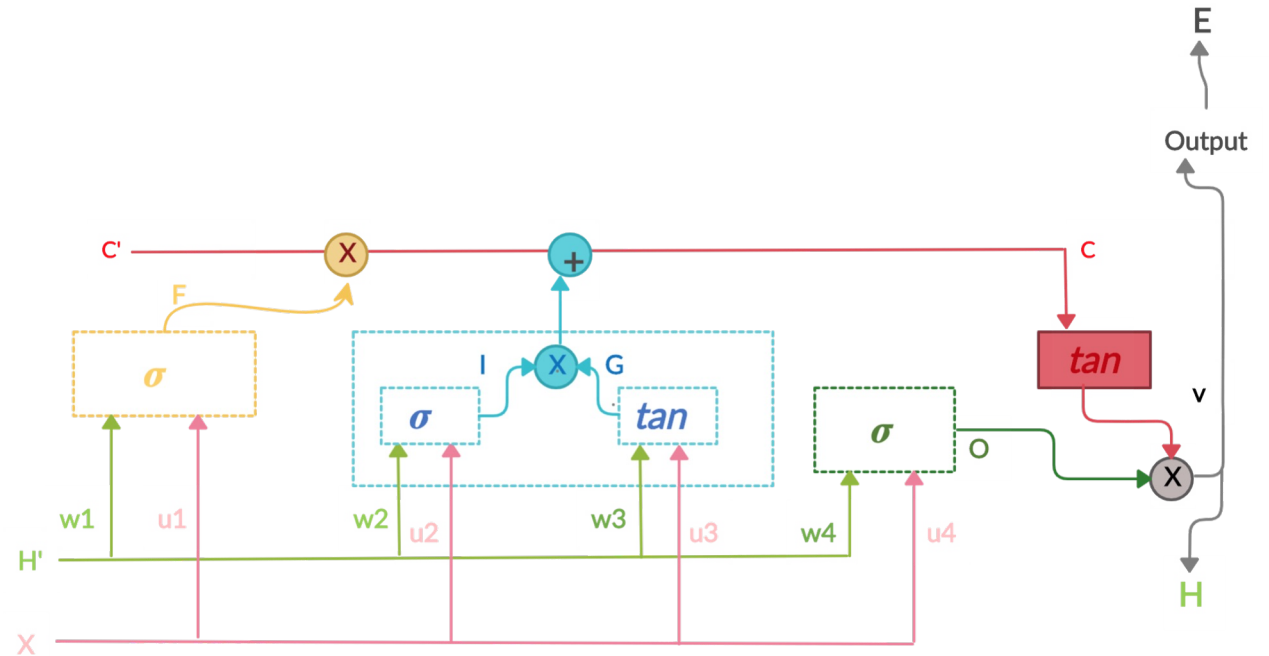
Forget Gate: Decide when some memory should be removed based on new information.

Input Gate: This gate decide if we need to update the memory based on new input.

Output Gate: output based on the inputs and the memory.



Math behind LSTM, Forward Prop.



X — Input

H' — Previous Layer's output

Output — Output of the network for the given X

H — Output to be fed to the next layer

E — Mean squared error between Y (predicted) and Output (actual)

Math behind LSTM,

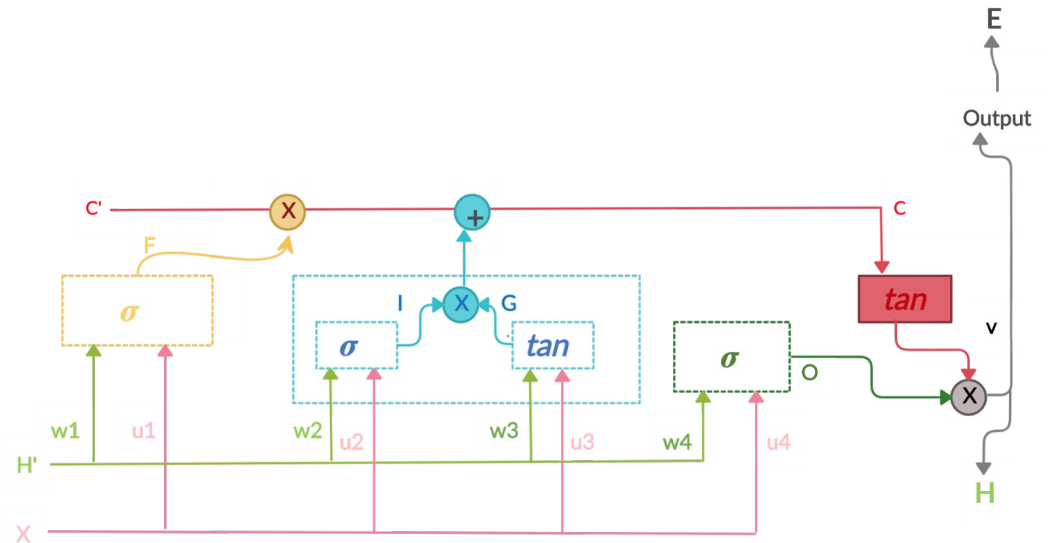
Forward Prop.

Forget Gate: $F = \sigma(u_1X + w_1H')$

Input Gate: $I = \sigma(u_2X + w_2H')$
 $G = \tan(u_3X + w_3H')$

Memory: $C = C'F + IG$

Output: $O = \sigma(u_4X + w_4H')$
 $H' = O \cdot \tan(C)$
 $Output = vH'$



Math behind LSTM,

Backward Prop.

As we have discussed, we do backward prop to tune the model weights, how this is done for an LSTM unit?

Lets consider one LSTM unit and ignore the biases for simplicity.

$$E = \frac{(\text{Output} - Y)^2}{2}$$

$$\frac{\partial E}{\partial u_1} = ?, \frac{\partial E}{\partial u_2} = ?, \frac{\partial E}{\partial u_3} = ?, \frac{\partial E}{\partial u_4} = ?$$

$$\frac{\partial E}{\partial w_1} = ?, \frac{\partial E}{\partial w_2} = ?, \frac{\partial E}{\partial w_3} = ?, \frac{\partial E}{\partial w_4} = ?$$

$$\frac{\partial E}{\partial v} = ?$$

Math behind LSTM,

Backward Prop.

Using the chain rule we can calculate the differential equations.

The equations are presented here just to get the idea.

$$\begin{aligned}\frac{\partial E}{\partial u_1} &= \frac{\partial E}{\partial Output} \frac{\partial Output}{\partial H'} \frac{\partial H'}{\partial C} \frac{\partial C}{\partial F} \frac{\partial F}{\partial u_1} \\ &= (Output - Y).v. \left(O \left(\sec^2 C \right) \right). (C'). (\sigma(u_1 X + w_1 H')). (1 - \sigma(u_1 X + w_1 H')). X\end{aligned}\quad (1)$$

$$\begin{aligned}\frac{\partial E}{\partial u_2} &= \frac{\partial E}{\partial Output} \frac{\partial Output}{\partial H'} \frac{\partial H'}{\partial C} \frac{\partial C}{\partial I} \frac{\partial I}{\partial u_2} \\ &= (Output - Y).v. \left(O \left(\sec^2 C \right) \right). (G). (\sigma(u_2 X + w_2 H')). (1 - \sigma(u_2 X + w_2 H')). X\end{aligned}\quad (2)$$

$$\begin{aligned}\frac{\partial E}{\partial u_3} &= \frac{\partial E}{\partial Output} \frac{\partial Output}{\partial H'} \frac{\partial H'}{\partial C} \frac{\partial C}{\partial G} \frac{\partial G}{\partial u_3} \\ &= (Output - Y).v. \left(O \left(\sec^2 C \right) \right). (I). \left(\sec^2(u_3 X + w_3 H') \right) X\end{aligned}\quad (3)$$

$$\begin{aligned}\frac{\partial E}{\partial u_4} &= \frac{\partial E}{\partial Output} \frac{\partial Output}{\partial H'} \frac{\partial H'}{\partial O} \frac{\partial O}{\partial u_4} \\ &= (Output - Y).v. \left(\tan(C). \sec^2 C \right). (\sigma(u_4 X + w_4 H')). (1 - \sigma(u_4 X + w_4 H')). X\end{aligned}\quad (4)$$

Math behind LSTM,
Backward Prop.

$$\begin{aligned}\frac{\partial E}{\partial w_1} &= \frac{\partial E}{\partial Output} \frac{\partial Output}{\partial H'} \frac{\partial H'}{\partial C} \frac{\partial C}{\partial F} \frac{\partial F}{\partial w_1} \\ &= (Output - Y) \cdot v \cdot (O(\sec^2 C)) \cdot (C') \cdot (\sigma(u_1 X + w_1 H')) \cdot (1 - \sigma(u_1 X + w_1 H')) \cdot H'\end{aligned} \quad (5)$$

$$\begin{aligned}\frac{\partial E}{\partial w_2} &= \frac{\partial E}{\partial Output} \frac{\partial Output}{\partial H'} \frac{\partial H'}{\partial C} \frac{\partial C}{\partial I} \frac{\partial I}{\partial w_2} \\ &= (Output - Y) \cdot v \cdot (O(\sec^2 C)) \cdot (G) \cdot (\sigma(u_2 X + w_2 H')) \cdot (1 - \sigma(u_2 X + w_2 H')) \cdot H'\end{aligned} \quad (6)$$

$$\begin{aligned}\frac{\partial E}{\partial w_3} &= \frac{\partial E}{\partial Output} \frac{\partial Output}{\partial H'} \frac{\partial H'}{\partial C} \frac{\partial C}{\partial G} \frac{\partial G}{\partial w_3} \\ &= (Output - Y) \cdot v \cdot (O(\sec^2 C)) \cdot (I) \cdot (\sec^2(u_3 X + w_3 H')) \cdot H'\end{aligned} \quad (7)$$

$$\begin{aligned}\frac{\partial E}{\partial w_4} &= \frac{\partial E}{\partial Output} \frac{\partial Output}{\partial H'} \frac{\partial H'}{\partial O} \frac{\partial O}{\partial w_4} \\ &= (Output - Y) \cdot v \cdot (\tan(C) \cdot \sec^2 C) \cdot (\sigma(u_4 X + w_4 H')) \cdot (1 - \sigma(u_4 X + w_4 H')) \cdot H'\end{aligned} \quad (8)$$

$$\frac{\partial E}{\partial v} = \frac{\partial E}{\partial Output} \frac{\partial Output}{\partial v} = (Output - Y) \cdot H' \quad (9)$$

Final updated weights:

$$w_{new} = w_{old} - \frac{\partial E}{\partial w}, \quad v_{new} = v_{old} - \frac{\partial E}{\partial v}, \quad u_{new} = u_{old} - \frac{\partial E}{\partial u}$$

Dropouts:

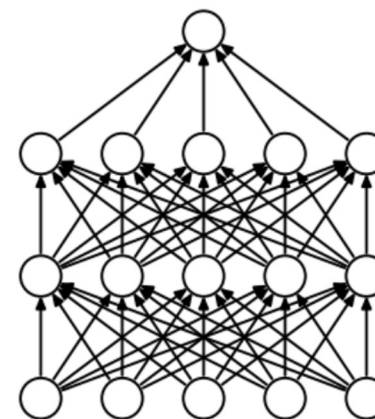
Generally speaking large NN (especially deep NN), may overfit the data specially if the training data is small in size.

One efficient way is **dropout**.

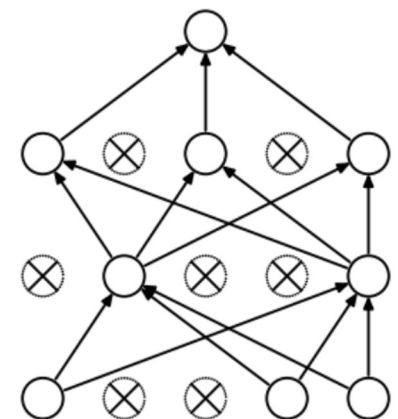
During training some of the nodes values are dropped out (temporary removed from network, along with all its connections in and out). This will cause the layer (and the NN) to be a different one.

Source of paper: Dropout: A Simple Way to Prevent Neural Networks from Overfitting

<http://jmlr.org/papers/v15/srivastava14a.html>



(a) Standard Neural Net



(b) After applying dropout.



Python: Hands on exercise:

Build a RNN to recognize human activity signals

Sample video: https://www.youtube.com/watch?v=XOEN9W05_4A

Categories of human activities:

- Walking,
- Walking stairs:
 - A) Upstairs
 - B) Downstairs
- Standing,
- Sitting,
- Laying,